



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

DATA STRUCTURE AND ALGORITHM SECJ2013

ASSIGNMENT 2: Real-World Application of Stack Using Array-Based and Pointer-Based Implementation in C++

**DUE DATE:
01/01/2025**

**GROUP:
8**

NAME	MATRIC NO
DAMIYA AINA BINTI BASIR ABD SHAMMAD	A23CS0220
LAM YOKE YU	A23CS0233
MUHAMMAD HAFIZI SYAMIR BIN HERMAN	A22EC0213
TAN YI YA	A23CS0187

TASK 1

1. Explain the basic operations of a Stack (Push, Pop, stackTop, IsEmpty, IsFull) with examples (in C++ comments).

A stack is a data structure that operates on the LIFO (Last-In-First-Out) principle, meaning the last element added is the first to be removed. The basic operations of a stack include the following:

1. Push happens when adding an element to the top of the stack.

Example: `stack.push(10); // Adds 10 to the top of the stack`

2. Pop happens when removing the element from the top of the stack.

Example: `stack.pop(); // Removes the top element from the stack`

3. StackTop happens when retrieving the top element without removing it.

Example: `int topElement = stack.top(); // Retrieves the element at the top`

4. IsEmpty happens when checking if the stack is empty. If there are no elements, it returns true; otherwise, it returns false.

Example: `if (stack.empty()) {
 // The stack is empty
}`

5. IsFull happens when checking if the stack has reached its maximum capacity. This is typically relevant in fixed-size implementations of stacks.

Example: `if (stack.size() == stack.max_size()) {
 // The stack is full
}`

2. Explain the LIFO (Last-In-First-Out) principle and its relevance in real-world scenarios.

The LIFO (Last-In-First-Out) principle is central to stack operations, where the last item added is the first to be removed. This can be compared to a stack of plates, where you always take the top plate first when removing items. The LIFO concept is widely used in practical scenarios and problem-solving.

For example, the Undo feature in text editors uses a stack to reverse the most recent changes, such as undoing the last typed character or action. Similarly, web browsers maintain a navigation history using stacks; visiting a new page adds it to the stack, and pressing the "Back" button removes the latest page to return to the previous one.

In programming, stacks are vital for managing function calls. When a function is invoked, its execution context is pushed onto a stack, and once the function completes, it is popped off, allowing the program to return to the previous state seamlessly. Additionally, stacks are utilized in algorithms for tasks like reversing strings or evaluating mathematical expressions in compilers and calculators.

The LIFO principle provides a straightforward and efficient method for handling processes where the order of actions is significant, making stacks invaluable in both computing and real-life applications.

TASK 2

The real world application that uses the concept of stack chosen is backtracking in algorithms in maze solving. Below are the process of maze solving alongside with the explanations of applications of stack in it.

1. The player, indicating the entity moving in the maze, starts at the entry point. The starting cell at the entry point is pushed onto the stack using the push operation.
2. All possible moves to adjacent cells are identified. If a valid move is found, push 1 of the adjacent cells into the stack, and the player moves into the new cell. The cell is marked as visited.
3. If no valid move is found, the top cell is popped from the stack, and the player will return to the previous cell, hence the word backtracking. Process 2 and 3 is repeatedly executed.
4. When the player reaches the exit cell, the top of the stack will contain the exit cell. The stack represents the solution to the maze.

Use cases of stackTop, IsEmpty, IsFull operations in maze solving are shown below.

1. stackTop operation is used to retrieve the player location without removing the top cell from the stack.
2. isEmpty operation is used for checking, that is if the stack is empty, it means all of the possible routes are explored and there is no solution for the maze.
3. isFull operation is also used for checking. If isFull operation returns true, it indicates the lack of memory to support the maze solving.

TASK 3

Both programs demonstrate the stack data structure to solve a maze using the backtracking algorithm. The stack is implemented in an array and in a linked-list. The maze is represented as a 2D array where 1 indicates open paths, and 0 represents obstacles. Predefined directions for rows and columns control movement within the maze, walking through all possible directions, up (0), right (1), down (2) or left (3).

Key operations of the `Stack` include:

- `createStack`: create the stack and initialise values
- `push`: adds a new position to the stack
- `pop`: removes the top position
- `stackTop`: retrieves the top position without removing it
- `isEmpty`: checks if the stack is empty
- `isFull`: checks if the stack has reached its maximum capacity (applied to array-based implementation only)

The `solveMaze` function houses the solving algorithm. The algorithm starts by pushing the starting position onto the stack and marking it as visited. It iteratively explores adjacent cells, pushing valid moves onto the stack. If a dead end is reached (no valid moves), the algorithm backtracks by popping the top position from the stack. This process continues until the exit cell is reached or the stack becomes empty, indicating no solution exists. The stack stores the path in reverse order, so a temporary array-based stack is used to display the path from start to exit in the correct sequence.

The functionalities and logic are similar in both implementations. The difference is the stack implementation. In the array-based implementation, the `Stack` class houses the index to keep track of the top index and store the fixed size array. In the pointer-based implementation, the `Stack` class houses the pointer to the top of the stack. A `Node` class is also created to store the data of the position and the next pointer.

Array-Based Implementation:

```
#include <iostream>
using namespace std;

const int SIZE = 100;

// N = ROWS, M = COLS
const int N = 5, M = 5;

class Stack{
private:
    int top;
    int data[SIZE][3];

public:
    void createStack() {
        top = -1;
    }
}
```

```

void push(int i, int j, int d){
    if (isFull()){
        cout << "Cannot push item. Stack is full!\n";
    }
    else{
        top = top + 1;
        data [top][0] = i;
        data [top][1] = j;
        data [top][2] = d;
    }
}

void pop(){
    int i, j;
    if(isEmpty()){
        cout << "Cannot pop item. Stack is empty!\n";
    }
    else{
        top = top - 1;
    }
}

int* stackTop(){
    if (isEmpty()){
        // cout << "Stack is empty!\n";
        return nullptr;
    }
    else{
        return data[top];
    }
}

void display(){
    char solution[N][M] = {
        {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
        {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
        {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
        {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
        {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '}};

    for (int t = top; t >= 0; t--){
        int i = data[t][0];
        int j = data[t][1];
        int d = data[t][2];
        cout << "(" << i << ", " << j << ")";
        if(t){
            cout << " -> ";
        }
        if(d == 0) solution[i][j] = '^';
        else if(d == 1) solution[i][j] = '>';
        else if(d == 2) solution[i][j] = 'v';
        else if(d == 3) solution[i][j] = '<';
    }
    cout << endl << endl;

    for (int i = 0; i < N; i++){
        cout << "|";
        for (int j = 0; j < M; j++){
            cout << solution[i][j] << "|";

```

```

        }
        cout << endl;
    }
    cout << endl;
}
bool isFull() {
    return (top == SIZE - 1);
}
bool isEmpty() {
    return (top == -1);
}
};

void printMaze(int maze[N][M]) {
    cout << "<<<< Current Maze >>>>\n";
    for (int i = 0; i < N; i++) {
        cout << "|";
        for (int j = 0; j < M; j++) {
            cout << maze[i][j] << "|";
        }
        cout << endl;
    }
    cout << endl;
}

void customiseMaze(int maze[N][M], int* starti, int* startj, int*
endi, int* endj) {
    cout
    << "This is a " << M << " * " << N << " maze.\n"
    << "Use 1 to indicate a valid path and 0 for an invalid
path\n"
    << "Enter a value followed by a space\n\n";

    for (int i = 0; i < N; i++) {
        cout << "Row #" << i << ": ";
        for (int j = 0; j < M; j++) {
            cin >> maze[i][j];
        }
    }

    printMaze(maze);

    cout
    << "Indicate the starting point and ending point.\n"
    << "Top left coordinate: (0, 0)\n"
    << "Top right coordinate: (" << 0 << ", " << M-1 << ")\n\n";

    cout << "Starting coordinate-i: ";
    cin >> *starti;
    while(*starti >= N || *starti < 0) {
        cout << "Invalid value\n";
        cout << "Starting coordinate-i: ";
        cin >> *starti;
    }
    cout << "Starting coordinate-j: ";
    cin >> *startj;
}

```

```

while(*startj >= M || *startj < 0){
    cout << "Invalid value\n";
    cout << "Starting coordinate-j: ";
    cin >> *startj;
}
cout << "Exit coordinate-i: ";
cin >> *endi;
while(*endi >= N || *endi < 0){
    cout << "Invalid value\n";
    cout << "Exit coordinate-i: ";
    cin >> *endi;
}
cout << "Exit coordinate-j: ";
cin >> *endj;
while(*endj >= M || *endj < 0){
    cout << "Invalid value\n";
    cout << "Exit coordinate-y: ";
    cin >> *endj;
}

cout << endl;
}

bool solveMaze(Stack* path, int maze[N][M], int endi = N-1, int
endj = M-1){
    bool visited[N][M] = {0};
    bool pathFound = false;

    while(!path->isEmpty()){
        int* temp = path->stackTop();
        int i = temp[0];
        int j = temp[1];
        int d = temp[2];

        // If we reach the end
        if (i == endi && j == endj){
            cout << "Path Found\n";
            path->display();
            return true;
        }

        // Checking each direction
        if(d == 0){ // Up
            if(i - 1 >= 0 && maze[i - 1][j] && !visited[i -
1][j]){
                visited[i - 1][j] = true;
                path->push(i - 1, j, 0);
            }
            else{
                path->pop();
                path->push(i, j, 1);
            }
        }
        else if(d == 1){ // Right
            if(j + 1 < M && maze[i][j + 1] && !visited[i][j + 1]){
                visited[i][j + 1] = true;

```



```

        path->push(i, j + 1, 0);
    }
    else{
        path->pop();
        path->push(i, j, 2);
    }
}
else if(d == 2){    // Down
    if(i + 1 < N && maze[i + 1][j] && !visited[i + 1][j]){
        visited[i + 1][j] = true;
        path->push(i + 1, j, 0);
    }
    else{
        path->pop();
        path->push(i, j, 3);
    }
}
else if(d == 3){    // Left
    if(j - 1 >= 0 && maze[i][j - 1] && !visited[i][j -
1]){

        visited[i][j - 1] = true;
        path->push(i, j - 1, 0);
    }
    else{
        path->pop();
        path->push(i, j, 4);
    }
}
else{
    visited[i][j] = true;
    path->pop();
}
}

cout << "Path Not Found\n\n";
return false;
}

int main(){
    cout
    << "***** Maze Solving Program *****\n"
    << "The maze is solved using backtracking algorithm.\n"
    << "The algorithm is implemented using array-based stack.\n"
    << "Stack operations are implemented in solveMaze().\n\n";

    int defaultMaze[N][M] = {
        {1, 1, 1, 1, 0},
        {1, 0, 0, 1, 0},
        {1, 1, 0, 1, 0},
        {1, 0, 0, 1, 0},
        {1, 1, 1, 1, 1}};

    int customMaze[N][M] = {
        {0, 0, 0, 0, 0},
        {0, 0, 0, 0, 0},
        {0, 0, 0, 0, 0},

```

```

        {0, 0, 0, 0, 0},
        {0, 0, 0, 0, 0}};

int maze[N][M];
for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
        maze[i][j] = defaultMaze[i][j];
    }
}

// Start and End coordinates
int starti = 0;
int startj = 0;
int endi = N-1;
int endj = M-1;

int choice = 1;

while (choice != 5){
    cout
    << "<<<< Menu >>>>\n"
    << "1. Print maze\n"
    << "2. Use Customise maze\n"
    << "3. Use Default maze\n"
    << "4. Solve maze\n"
    << "5. Quit\n"
    << "Choice: ";
    cin >> choice;

    cout << endl;

    if(choice == 1){
        printMaze(maze);
    }
    else if(choice == 2){
        cout << "<<<< Customise Maze >>>>\n";
        customiseMaze(customMaze, &starti, &startj, &endi,
&endj);

        for (int i = 0; i < N; i++) {
            for (int j = 0; j < M; j++) {
                maze[i][j] = customMaze[i][j];
            }
        }
    }
    else if(choice == 3){

        for (int i = 0; i < N; i++) {
            for (int j = 0; j < M; j++) {
                maze[i][j] = defaultMaze[i][j];
            }
        }
        starti = 0;
        startj = 0;
        endi = N-1;
        endj = M-1;
    }
}

```

```

        else if(choice == 4){
            // Create and push start to stack
            Stack path;
            path.createStack();
            path.push(starti, startj, 0);

            cout << "Solving Maze...\n";
            bool pathFound = solveMaze(&path, maze, endi, endj);
        }
        else if(choice == 5){
            cout << "Terminating...\n";
            return 0;
        }
    }
}

```

Array-Based Implementation Output:

***** Maze Solving Program *****
 The maze is solved using backtracking algorithm.
 The algorithm is implemented using array-based stack.
 Stack operations are implemented in solveMaze().

<<<< Menu >>>>

1. Print maze
2. Use Customise maze
3. Use Default maze
4. Solve maze
5. Quit

Choice: 1

<<<< Current Maze >>>>

```

|1|1|1|1|0|
|1|0|0|1|0|
|1|1|0|1|0|
|1|0|0|1|0|
|1|1|1|1|1|

```

<<<< Menu >>>>

1. Print maze
2. Use Customise maze
3. Use Default maze
4. Solve maze
5. Quit

Choice: 2

<<<< Customise Maze >>>>

This is a 5 * 5 maze.

Use 1 to indicate a valid path and 0 for an invalid path

Enter a value followed by a space

```

Row #0: 1 1 0 1 1
Row #1: 1 0 0 0 0
Row #2: 1 1 0 0 1
Row #3: 1 0 1 0 1

```

```
Row #4: 1 1 0 1 1
<<<<< Current Maze >>>>>
|1|1|0|1|1|
|1|0|0|0|0|
|1|1|0|0|1|
|1|0|1|0|1|
|1|1|0|1|1|
```

Indicate the starting point and ending point.
Top left coordinate: (0, 0)
Top right coordinate: (0, 4)

```
Starting coordinate-i: 0
Starting coordinate-j: 0
Exit coordinate-i: 4
Exit coordinate-j: 4
```

```
<<<<< Menu >>>>>
1. Print maze
2. Use Customise maze
3. Use Default maze
4. Solve maze
5. Quit
Choice: 4
```

Solving Maze...
Path Not Found

```
<<<<< Menu >>>>>
1. Print maze
2. Use Customise maze
3. Use Default maze
4. Solve maze
5. Quit
Choice: 3
```

```
<<<<< Menu >>>>>
1. Print maze
2. Use Customise maze
3. Use Default maze
4. Solve maze
5. Quit
Choice: 1
```

```
<<<<< Current Maze >>>>>
|1|1|1|1|0|
|1|0|0|1|0|
|1|1|0|1|0|
|1|0|0|1|0|
|1|1|1|1|1|
```

```
<<<<< Menu >>>>>
1. Print maze
2. Use Customise maze
3. Use Default maze
4. Solve maze
```

5. Quit

Choice: 4

Solving Maze...

Path Found

(4, 4) -> (4, 3) -> (3, 3) -> (2, 3) -> (1, 3) -> (0, 3) -> (0, 2)
-> (0, 1) -> (0, 0)

```
|>|>|>|v| |
| | | |v| |
| | | |v| |
| | | |v| |
| | | |v| |
| | | |>|^|
```

<<<< Menu >>>>

1. Print maze
2. Use Customise maze
3. Use Default maze
4. Solve maze
5. Quit

Choice: 5

Terminating...

Pointer-Based Implementation:

```
#include <iostream>
using namespace std;

const int SIZE = 50;

// N = ROWS, M = COLS
const int N = 5, M = 5;

class Node{
public:
    int i, j, d;
    Node* next;

    Node(int a = 0, int b = 0, int c = 0, Node* nxt = nullptr){
        i = a;
        j = b;
        d = c;
        next = nxt;
    }
};

class Stack{
private:
    Node* head;

public:
    void createStack(){
        head = nullptr;
    }
};
```

```

void push(int i, int j, int d){
    Node* newNode = new Node(i, j, d, head);
    head = newNode;
}
void pop(){
    if(isEmpty()){
        cout << "Cannot pop item. Stack is empty!\n";
    }
    else{
        Node* toDelete = head;
        head = head->next;
        delete toDelete;
    }
}
Node* stackTop(){
    if (isEmpty()){
        // cout << "Stack is empty!\n";
        return nullptr;
    }
    else{
        return head;
    }
}
void display(){
    char solution[N][M] = {
        {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
        {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
        {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
        {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '},
        {' ', ' ', ' ', ' ', ' ', ' ', ' ', ' ', ' '}};

    Node* current = head;

    while(current){
        int i = current->i;
        int j = current->j;
        int d = current->d;

        cout << "(" << i << ", " << j << ")";
        if(current->next){
            cout << " -> ";
        }

        if(d == 0) solution[i][j] = '^';
        else if(d == 1) solution[i][j] = '>';
        else if(d == 2) solution[i][j] = 'v';
        else if(d == 3) solution[i][j] = '<';

        current = current->next;
    }
    cout << endl << endl;

    for (int i = 0; i < N; i++){
        cout << "|";
        for (int j = 0; j < M; j++){
            cout << solution[i][j] << "|";

```

```

        }
        cout << endl;
    }
    cout << endl;
}
bool isEmpty() {
    return (head == nullptr);
}
};

void printMaze(int maze[N][M]) {
    cout << "<<<< Current Maze >>>>\n";
    for (int i = 0; i < N; i++) {
        cout << "|";
        for (int j = 0; j < M; j++) {
            cout << maze[i][j] << "|";
        }
        cout << endl;
    }
    cout << endl;
}

void customiseMaze(int maze[N][M], int* starti, int* startj, int*
endi, int* endj) {
    cout
    << "This is a " << M << " * " << N << " maze.\n"
    << "Use 1 to indicate a valid path and 0 for an invalid
path\n"
    << "Enter a value followed by a space\n\n";

    for (int i = 0; i < N; i++) {
        cout << "Row #" << i << ": ";
        for (int j = 0; j < M; j++) {
            cin >> maze[i][j];
        }
    }

    printMaze(maze);

    cout
    << "Indicate the starting point and ending point.\n"
    << "Top left coordinate: (0, 0)\n"
    << "Top right coordinate: (" << 0 << ", " << M-1 << ")\n\n";

    cout << "Starting coordinate-i: ";
    cin >> *starti;
    while(*starti >= N || *starti < 0) {
        cout << "Invalid value\n";
        cout << "Starting coordinate-i: ";
        cin >> *starti;
    }
    cout << "Starting coordinate-j: ";
    cin >> *startj;
    while(*startj >= M || *startj < 0) {
        cout << "Invalid value\n";
        cout << "Starting coordinate-j: ";

```

```

        cin >> *startj;
    }
    cout << "Exit coordinate-i: ";
    cin >> *endi;
    while(*endi >= N || *endi < 0){
        cout << "Invalid value\n";
        cout << "Exit coordinate-i: ";
        cin >> *endi;
    }
    cout << "Exit coordinate-j: ";
    cin >> *endj;
    while(*endj >= M || *endj < 0){
        cout << "Invalid value\n";
        cout << "Exit coordinate-y: ";
        cin >> *endj;
    }

    cout << endl;
}

bool solveMaze(Stack* path, int maze[N][M], int endi = N-1, int
endj = M-1){
    bool visited[N][M] = {0};
    visited[0][0] = true;
    bool pathFound = false;

    while(!path->isEmpty()){
        Node* temp = path->stackTop();
        int i = temp->i;
        int j = temp->j;
        int d = temp->d;

        // If we reach the end
        if (i == endi && j == endj){
            cout << "Path Found\n";
            path->display();
            return true;
        }

        // Checking each direction
        if(d == 0){ // Up
            if(i - 1 >= 0 && maze[i - 1][j] && !visited[i -
1][j]){
                visited[i - 1][j] = true;
                path->push(i - 1, j, 0);
            }
            else{
                path->pop();
                path->push(i, j, 1);
            }
        }
        else if(d == 1){ // Right
            if(j + 1 < M && maze[i][j + 1] && !visited[i][j + 1]){
                visited[i][j + 1] = true;
                path->push(i, j + 1, 0);
            }
        }
    }
}

```



```

        else{
            path->pop();
            path->push(i, j, 2);
        }
    }
    else if(d == 2){        // Down
        if(i + 1 < N && maze[i + 1][j] && !visited[i + 1][j]){
            visited[i + 1][j] = true;
            path->push(i + 1, j, 0);
        }
        else{
            path->pop();
            path->push(i, j, 3);
        }
    }
    else if(d == 3){        // Left
        if(j - 1 >= 0 && maze[i][j - 1] && !visited[i][j -
1]){

            visited[i][j - 1] = true;
            path->push(i, j - 1, 0);
        }
        else{
            path->pop();
            path->push(i, j, 4);
        }
    }
    else{
        visited[i][j] = true;
        path->pop();
    }
}

cout << "Path Not Found\n\n";
return false;
}

int main(){
    cout
    << "***** Maze Solving Program *****\n"
    << "The maze is solved using backtracking algorithm.\n"
    << "The algorithm is implemented using pointer-based stack.\n"
    << "Stack operations are implemented in solveMaze().\n\n";

    int defaultMaze[N][M] = {
        {1, 1, 1, 1, 0},
        {1, 0, 0, 1, 0},
        {1, 1, 0, 1, 0},
        {1, 0, 0, 1, 0},
        {1, 1, 1, 1, 1}};

    int customMaze[N][M] = {
        {0, 0, 0, 0, 0},
        {0, 0, 0, 0, 0},
        {0, 0, 0, 0, 0},
        {0, 0, 0, 0, 0},
        {0, 0, 0, 0, 0}};

```

```

int maze[N][M];
for (int i = 0; i < N; i++) {
    for (int j = 0; j < M; j++) {
        maze[i][j] = defaultMaze[i][j];
    }
}

// Start and End coordinates
int starti = 0;
int startj = 0;
int endi = N-1;
int endj = M-1;

int choice = 1;

while (choice != 5) {
    cout
    << "<<<<< Menu >>>>>\n"
    << "1. Print maze\n"
    << "2. Use Customise maze\n"
    << "3. Use Default maze\n"
    << "4. Solve maze\n"
    << "5. Quit\n"
    << "Choice: ";
    cin >> choice;

    cout << endl;

    if(choice == 1){
        printMaze(maze);
    }
    else if(choice == 2){
        cout << "<<<<< Customise Maze >>>>>\n";
        customiseMaze(customMaze, &starti, &startj, &endi,
&endj);
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < M; j++) {
                maze[i][j] = customMaze[i][j];
            }
        }
    }
    else if(choice == 3){
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < M; j++) {
                maze[i][j] = defaultMaze[i][j];
            }
        }
        starti = 0;
        startj = 0;
        endi = N-1;
        endj = M-1;
    }
    else if(choice == 4){
        // Create and push start to stack

```

```

        Stack path;
        path.createStack();
        path.push(starti, startj, 0);

        cout << "Solving Maze...\n";
        bool pathFound = solveMaze(&path, maze, endi, endj);
    }
    else if(choice == 5){
        cout << "Terminating...\n";
        return 0;
    }
}
}

```

Pointer-Based Implementation Output:

***** Maze Solving Program *****
 The maze is solved using backtracking algorithm.
 The algorithm is implemented using pointer-based stack.
 Stack operations are implemented in solveMaze().

<<<< Menu >>>>

1. Print maze
 2. Use Customise maze
 3. Use Default maze
 4. Solve maze
 5. Quit
 Choice: 1

<<<< Current Maze >>>>

```

|1|1|1|1|0|
|1|0|0|1|0|
|1|1|0|1|0|
|1|0|0|1|0|
|1|1|1|1|1|

```

<<<< Menu >>>>

1. Print maze
 2. Use Customise maze
 3. Use Default maze
 4. Solve maze
 5. Quit
 Choice: 2

<<<< Customise Maze >>>>

This is a 5 * 5 maze.
 Use 1 to indicate a valid path and 0 for an invalid path
 Enter a value followed by a space

```

Row #0: 1 1 1 1 1
Row #1: 1 1 1 1 1
Row #2: 0 0 0 0 0
Row #3: 1 1 1 1 1

```

```
Row #4: 1 1 1 1 1
<<<< Current Maze >>>>
|1|1|1|1|1|
|1|1|1|1|1|
|0|0|0|0|0|
|1|1|1|1|1|
|1|1|1|1|1|
```

Indicate the starting point and ending point.
Top left coordinate: (0, 0)
Top right coordinate: (0, 4)

```
Starting coordinate-i: 0
Starting coordinate-j: 0
Exit coordinate-i: 4
Exit coordinate-j: 4
```

```
<<<< Menu >>>>
1. Print maze
2. Use Customise maze
3. Use Default maze
4. Solve maze
5. Quit
Choice: 4
```

Solving Maze...
Path Not Found

```
<<<< Menu >>>>
1. Print maze
2. Use Customise maze
3. Use Default maze
4. Solve maze
5. Quit
Choice: 3
```

```
<<<< Menu >>>>
1. Print maze
2. Use Customise maze
3. Use Default maze
4. Solve maze
5. Quit
Choice: 1
```

```
<<<< Current Maze >>>>
|1|1|1|1|0|
|1|0|0|1|0|
|1|1|0|1|0|
|1|0|0|1|0|
|1|1|1|1|1|
```

```
<<<< Menu >>>>
1. Print maze
2. Use Customise maze
3. Use Default maze
4. Solve maze
```

```

5. Quit
Choice: 4

Solving Maze...
Path Found
(4, 4) -> (4, 3) -> (3, 3) -> (2, 3) -> (1, 3) -> (0, 3) -> (0, 2)
-> (0, 1) -> (0, 0)

|>|>|>|v| |
| | | |v| |
| | | |v| |
| | | |v| |
| | | |v| |
| | | |>|^|

<<<<< Menu >>>>>
1. Print maze
2. Use Customise maze
3. Use Default maze
4. Solve maze
5. Quit
Choice: 5

Terminating...

```

TASK 4

A stack can be implemented using both array-based and pointer-based approaches. These two implementations are compared in terms of memory usage, ease of implementation and scalability.

Memory Usage

In array-based implementation, memory is allocated at the time of declaration with a fixed size. This can lead to memory wastage if the stack is not fully utilised.

Pointer-based implementation dynamically allocates memory for each element when needed, which is more memory-efficient. However, it consumes extra memory to keep the pointer of the next node. As the stack grows larger, the memory overhead for pointers can become significant, impacting overall memory efficiency.

Ease of Implementation

An array-based implementation is more straightforward to implement. An index is used to track the top element of the stack. During a push operation, the array is modified, and the index is incremented. During a pop operation, the index is decremented.

Pointer-based implementation is more complex. A pointer is used to track the top of the stack. During a push operation, a new node is created, housing the value and the pointer to the next node. The top pointer is then updated to the new node. During a pop operation, the top pointer is updated and the memory for the node at the top is freed.

Scalability

Since the pointer dynamically allocates memory, it is more scalable. This allows the stack to grow and shrink as needed. Array-based stacks require resizing if the stack exceeds its allocated size and resizing is costly as it requires copying data to a new array.

In short, array-based implementation is simple and memory-efficient for small stacks, but not flexible when the stack size needs to increase. Pointer-based implementation is more scalable, but may be complex due to the pointer implementation.